

Session 2 - Lab Programs

Name: H ASRAAR AHMED

Reg no: 192524028

Course name: DDA

Course code: CSA1420

Program 1

Question

Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Program Code

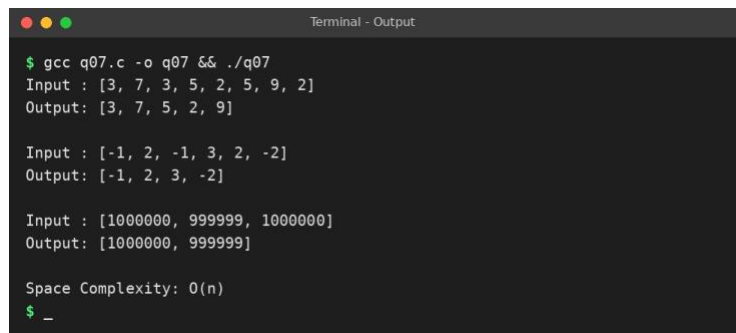
```
#include <stdio.h>
#include <stdlib.h>

/* Hash set with linear probing: Time  $O(n)$  average, Space  $O(n)$  */
int uniqueElements(int a[], int n, int out[]) {
    int cap = 2 * n + 1;
    int *table = malloc(cap * sizeof(int));
    int *used = calloc(cap, sizeof(int));
    int k = 0;
    for (int i = 0; i < n; i++) {
        long h = ((long)a[i] % cap + cap) % cap;
        while (used[h] && table[h] != a[i]) h = (h + 1) % cap;
        if (!used[h]) {
            /* first time we see this value */
            used[h] = 1;
            table[h] = a[i];
            out[k++] = a[i];
        }
    }
    free(table);
    free(used);
    return k;
}

void run(int a[], int n) {
    int out[100];
    int k = uniqueElements(a, n, out);
    printf("Input : [");
    for (int i = 0; i < n; i++)
        printf("%d%s", a[i], i < n - 1 ? ", " : "");
    printf("]\nOutput: [");
    for (int i = 0; i < k; i++)
        printf("%d%s", out[i], i < k - 1 ? ", " : "");
    printf("]\n\n");
}

int main() {
    int t1[] = {3, 7, 3, 5, 2, 5, 9, 2};
    int t2[] = {-1, 2, -1, 3, 2, -2};
    int t3[] = {1000000, 999999, 1000000};
    run(t1, 8);
    run(t2, 6);
    run(t3, 3);
    printf("Space Complexity:  $O(n)$ \n");
    return 0;
}
```

Output Screenshot



```
Terminal - Output

$ gcc q07.c -o q07 && ./q07
Input : [3, 7, 3, 5, 2, 5, 9, 2]
Output: [3, 7, 5, 2, 9]

Input : [-1, 2, -1, 3, 2, -2]
Output: [-1, 2, 3, -2]

Input : [1000000, 999999, 1000000]
Output: [1000000, 999999]

Space Complexity:  $O(n)$ 
$ _
```

Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code.

Program Code

```
#include <stdio.h>

/* Bubble sort: Best  $O(n)$  (already sorted), Average/Worst  $O(n^2)$ , Space  $O(1)$  */
void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                int t = a[j];
                a[j] = a[j + 1];
                a[j + 1] = t;
                swapped = 1;
            }
        }
        if (!swapped) break;
    }
    /* array already sorted */
}
```

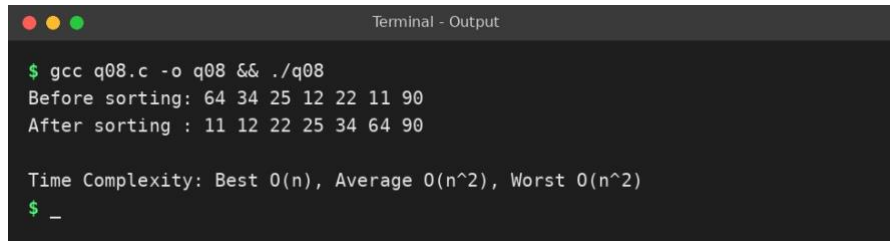
Program 2

Question

```
void print(const char *label, int a[], int n) {
    printf("%s", label);    for (int i = 0; i < n;
    i++) printf("%d ", a[i]);    printf("\n"); }

int main() {    int a[] = {64, 34, 25, 12, 22, 11, 90};    int n =
    sizeof(a) / sizeof(a[0]);    print("Before sorting: ", a, n);
    bubbleSort(a, n);    print("After sorting : ", a, n);    printf("\nTime
    Complexity: Best O(n), Average O(n^2), Worst O(n^2)\n");    return 0; }
```

Output Screenshot

A terminal window titled "Terminal - Output" showing the execution of a C program. The user enters the command to compile and run q08.c. The output shows the array before and after sorting, and the time complexity analysis.

```
Terminal - Output

$ gcc q08.c -o q08 && ./q08
Before sorting: 64 34 25 12 22 11 90
After sorting : 11 12 22 25 34 64 90

Time Complexity: Best O(n), Average O(n^2), Worst O(n^2)
$ _
```

Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

Program Code

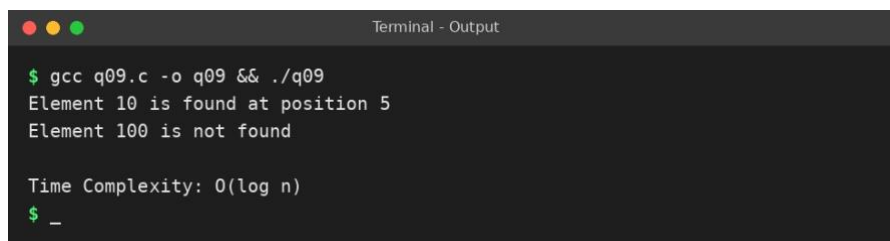
```
#include <stdio.h>

/* The sample array is not in sorted order, so we sort (value, original
index) pairs first, then binary search on the sorted values and report
the ORIGINAL 1-based position of the key. */ void sortWithIndex(int
val[], int idx[], int n) {    for (int i = 1; i < n; i++) {        /*
insertion sort on pairs */        int v = val[i], x = idx[i], j = i - 1;
while (j >= 0 && val[j] > v) {            val[j + 1] = val[j];
idx[j + 1] = idx[j];            j--;        }        val[j + 1] = v;
idx[j + 1] = x;    }
}

/* Binary search: Time O(log n), Space O(1). Returns index or -1
*/ int binarySearch(int arr[], int n, int key) {    int low = 0,
high = n - 1;    while (low <= high) {        int mid = low +
(high - low) / 2;        if (arr[mid] == key) return mid;
else if (arr[mid] < key) low = mid + 1;        else high = mid -
1;    }    return -1; }

void search(int X[], int n, int key) {    int val[20], idx[20];    for (int i =
0; i < n; i++) { val[i] = X[i]; idx[i] = i; }    sortWithIndex(val, idx, n);
int pos = binarySearch(val, n, key);    if (pos != -1) printf("Element %d is
found at position %d\n", key, idx[pos] + 1);    else        printf("Element %d
is not found\n", key); }

int main() {    int X[] = {3, 4, 6, -9,
10, 8, 9, 30};    int n = sizeof(X) /
sizeof(X[0]);    search(X, n, 10); }
```

A terminal window titled "Terminal - Output" showing the execution of a C program. The user enters the command to compile and run q09.c. The output shows the search results for the number 10 and 100, and the time complexity analysis.

```
Terminal - Output

$ gcc q09.c -o q09 && ./q09
Element 10 is found at position 5
Element 100 is not found

Time Complexity: O(log n)
$ _
```

Program 3

Question

```
search(X, n, 100);    printf("\nTime
Complexity: O(log n)\n");    return 0; }
```

Output Screenshot

Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

Program Code

```
#include <stdio.h>

/* Heap sort: Time  $O(n \log n)$  in all cases, Space  $O(1)$ , no built-in functions
*/ void swap(int *a, int *b) {    int t = *a;    *a = *b;
    *b = t;
}

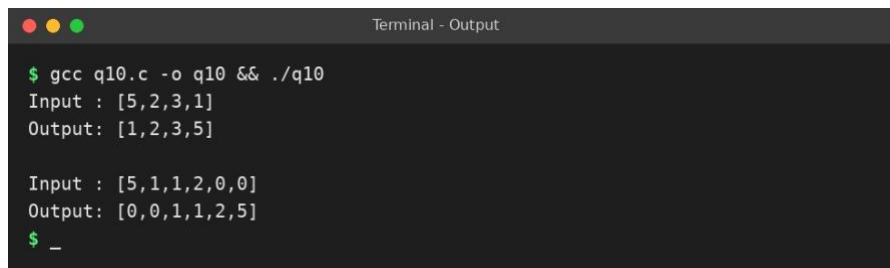
void heapify(int a[], int n, int i) {    int
largest = i, l = 2 * i + 1, r = 2 * i + 2;
if (l < n && a[l] > a[largest]) largest = l;
if (r < n && a[r] > a[largest]) largest = r;
if (largest != i) {        swap(&a[i],
&a[largest]);        heapify(a, n, largest);
} }

void sortArray(int a[], int n) {    for (int i = n / 2 - 1; i >=
0; i--)    /* build max heap */        heapify(a, n, i);    for
(int i = n - 1; i > 0; i--) {        /* move max to the end */
swap(&a[0], &a[i]);        heapify(a, i, 0);    } }

void run(int a[], int n) {    printf("Input : [");    for (int i = 0; i
< n; i++) printf("%d%s", a[i], i < n - 1 ? "," : "");    sortArray(a,
n);    printf("]\nOutput: [");    for (int i = 0; i < n; i++)
printf("%d%s", a[i], i < n - 1 ? "," : "");    printf("]\n\n"); }

int main() {    int t1[] = {5,
2, 3, 1};    int t2[] = {5, 1,
1, 2, 0, 0};    run(t1, 4);
run(t2, 6);    return 0; }
```

Output Screenshot



```
Terminal - Output

$ gcc q10.c -o q10 && ./q10
Input : [5,2,3,1]
Output: [1,2,3,5]

Input : [5,1,1,2,0,0]
Output: [0,0,1,1,2,5]
$ _
```